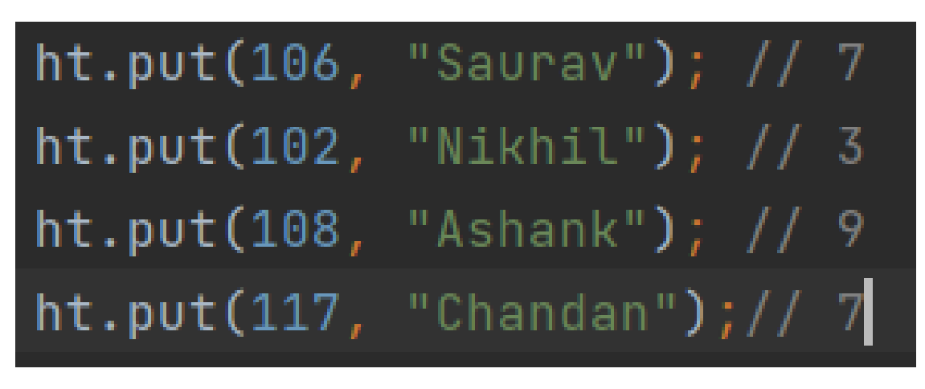
## STL Continued :→

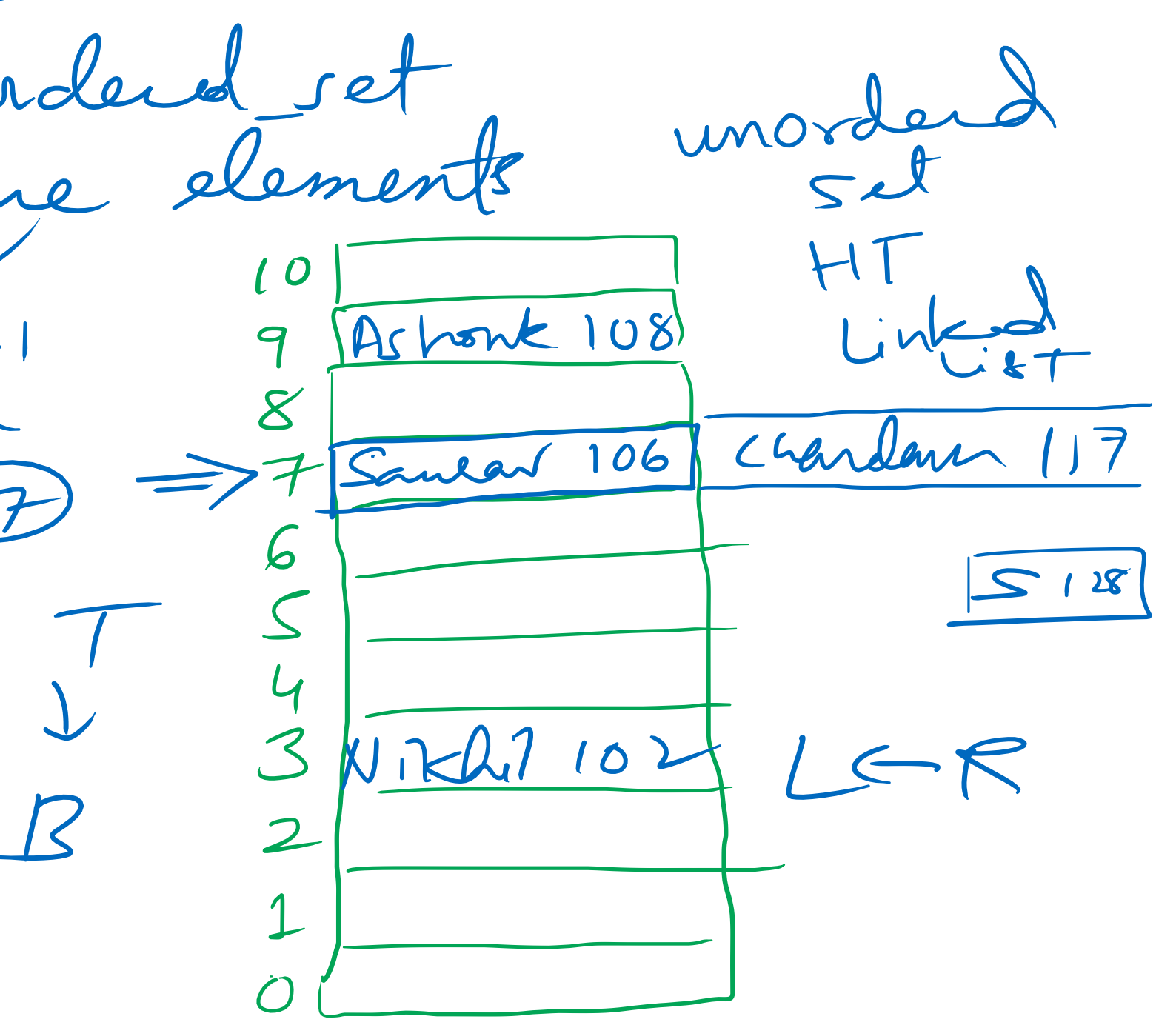
STL Sets : (i) ordered set or set
(ii) unordered_set
unique elements

Hashing Order

11 [128 121
Sahana (7)



```
ht.put(106, "Saurav"); // 7
ht.put(102, "Nikhil"); // 3
ht.put(108, "Ashank"); // 9
ht.put(117, "Chandan");// 7
```


unordered set
HT
Linked List
10
9 Ashank 108
8
7 Saurav 106
Chandan 117
S 128
6
5
4
3 Nikhil 102
2
1
0
T ↓ B
L←R

11 ⟌ 106 = 9, 99, (7)

11 ⟌ 102 = 9, 99, (3)

11 ⟌ 108 = 9, 99, (9)

11 ⟌ 117 = 10, 110, (7)

Top To Bottom
Right to Left
Hash Collision

## Difference between set & ordered-set

| Feature | set | unordered-set |
|---|---|---|
| Order | sorted | unspecified order |
| Underlying data structure | Balanced BST | Hash Table |
| Insertion time | O(log n) | O(1) |
| Deletion time | O( log n) | O (1) |
| Search time | O( log n) | O (1) |
| Use case | When order matters | What fast access matters and order doesn't matter |

## STL Vectors :→

Features :→

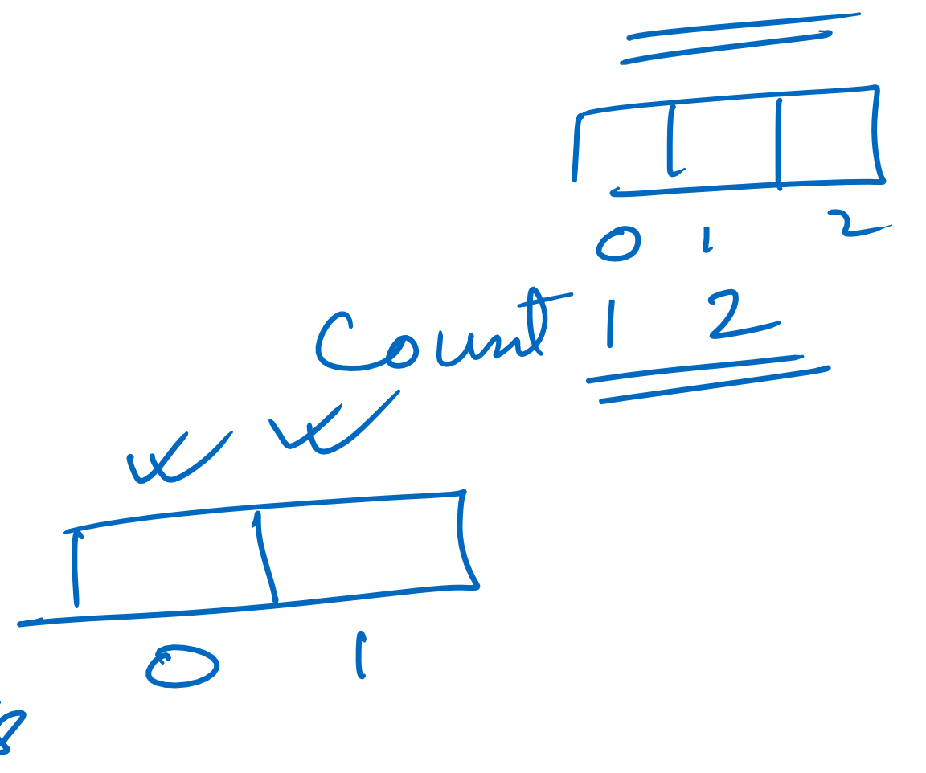
* Dynamic Sizing :→ Automatically resize

* Contiguous Storage :→ They are stored in contiguous memory, allowing for fast access using pointers/indices.
(0 to n-1)

* Automatic memory management :→
int * arr = new int[n] ;
→ in vectors its not required.
delete[] arr ; → not required.

Handles memory allocation & deallocation automatically.

vec3.erase(vec.begin() ; vec.begin()+2);
0
The last index is ignored
0+2


0 1 2
count 1 2
0 1

First Two Elements

v.data() = int *P | 7 | 6 | 2 | 1 | 3 |
0

arr = {7,6,2,1,3} 1000

P → 1000

*P → value (7)

arr → pointer = 1000

## The Heap Data Structure :

| 9 | 6 | 2 | 4 | 11 | 3 | 8 | 7 |
0 1 2 3 4 5 6 7

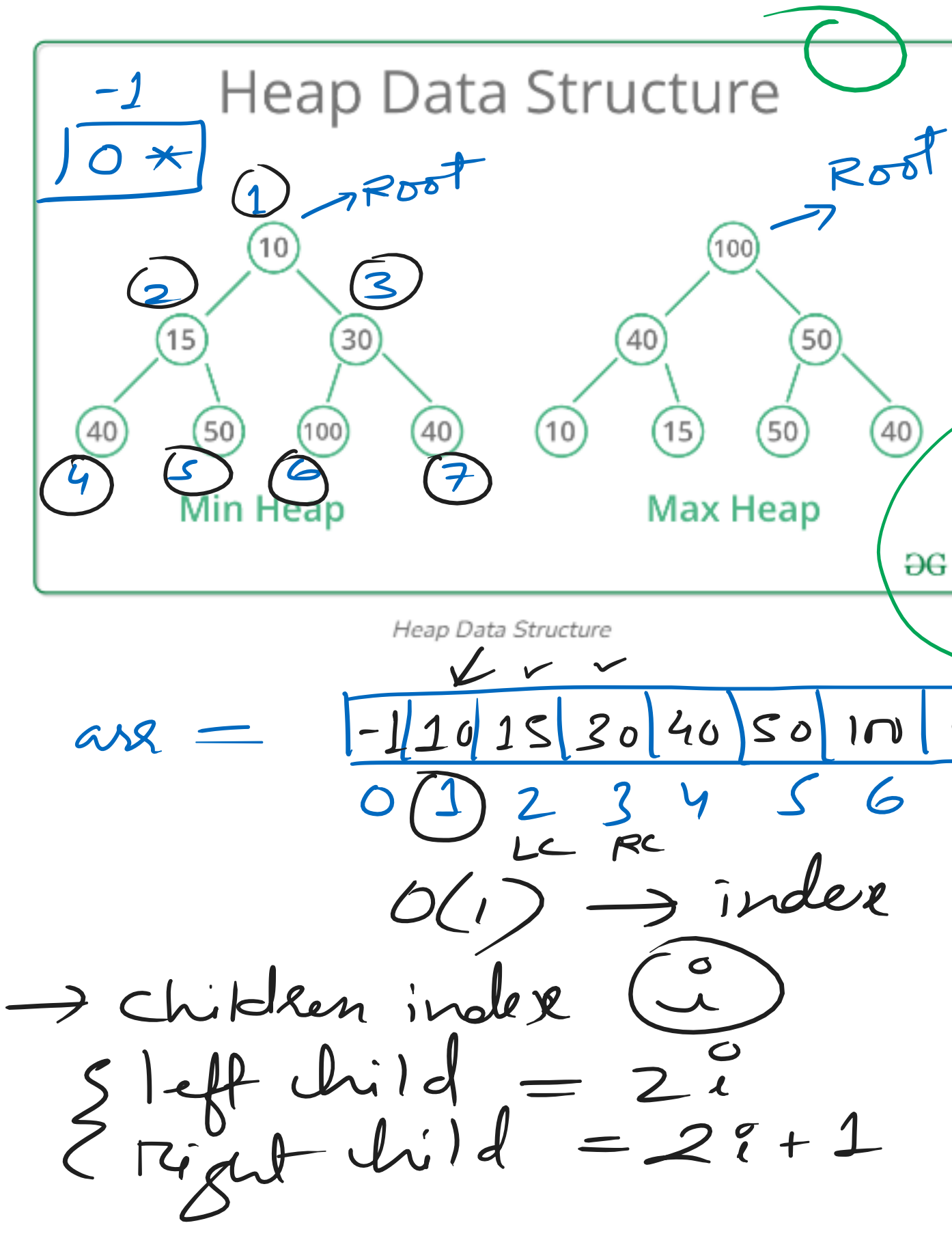
[n log n]
O(1)

I want the largest number : O(1)

I want the smallest number : O(1)

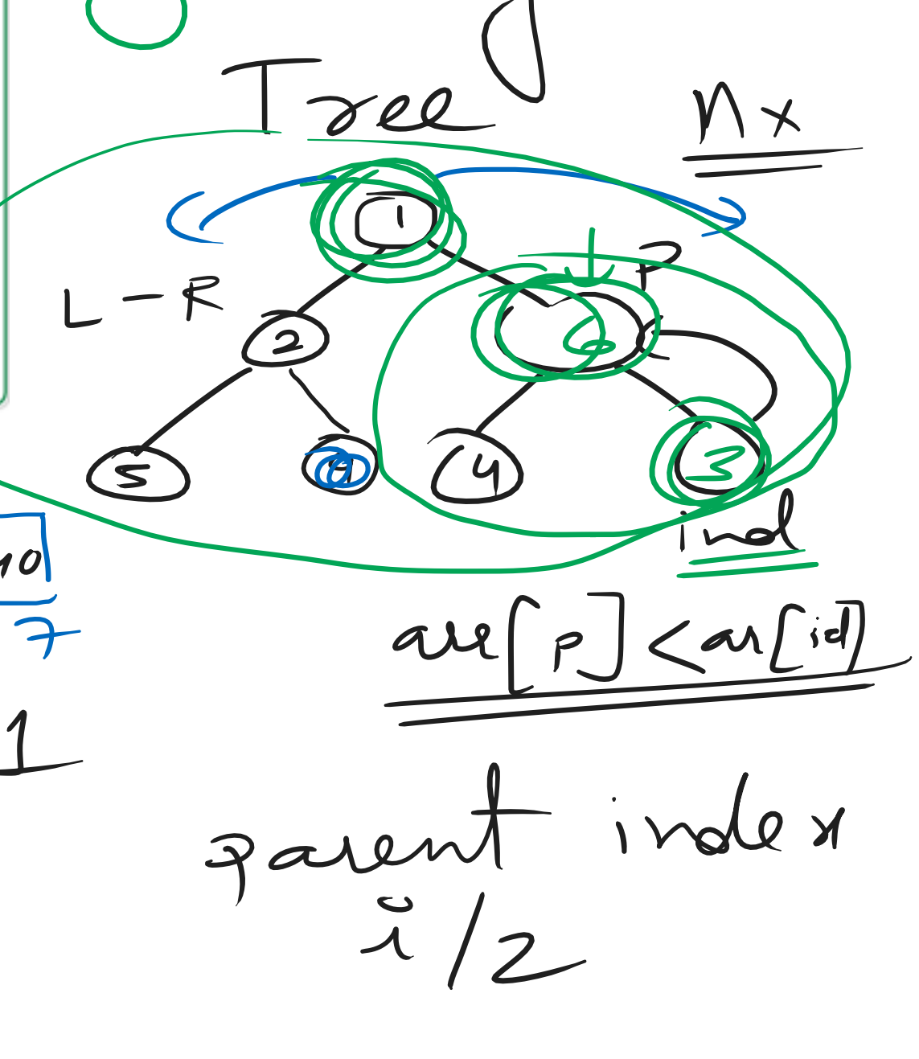
Linear Search → O(n)

Heaps < Max Heap, Min Heap


Heap Data Structure
-1
0 *
Root
Root
10
15
30
40
50
100
40
Min Heap
100
40
50
10
15
50
40
Max Heap
1
2
3
4
5
6
7

Heap Data Structure

⇒ Complete Binary Tree


nx
L-R
P
1
2
3
4
5
ind

arr[P] < arr[ind]

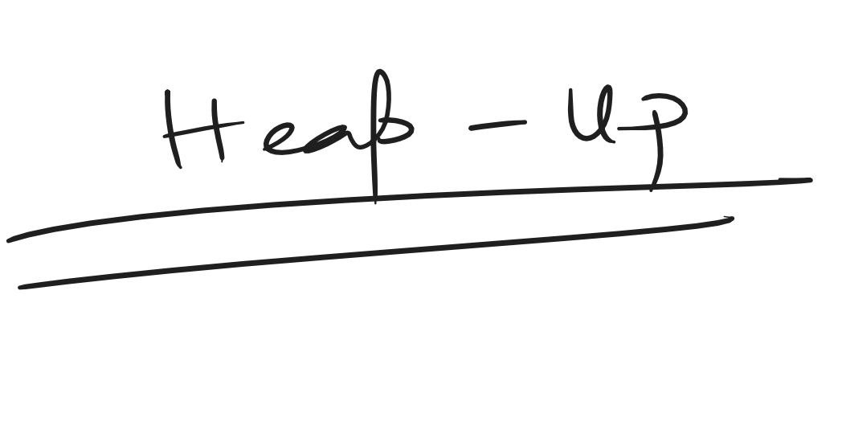
parent index
i/2

arr = | -1 | 10 | 15 | 30 | 40 | 50 | 100 | 40 |
0 (1) 2 3 4 5 6 7
LC RC

O(1) → index 1

→ children index (i)
{ left child = 2i
{ Right child = 2i+1

50, 55, 53, 52, 54


P
1
55
2
54
3
53
4
50
5
52

| 55 | 54 | 53 | 50 | 52 |
1 2 3 4 5

Heap - Up